

Ocar: Deployment & Infrastructure Architecture

Current state: network, compute, data layer, CI/CD, and the blue/green deploy mechanism

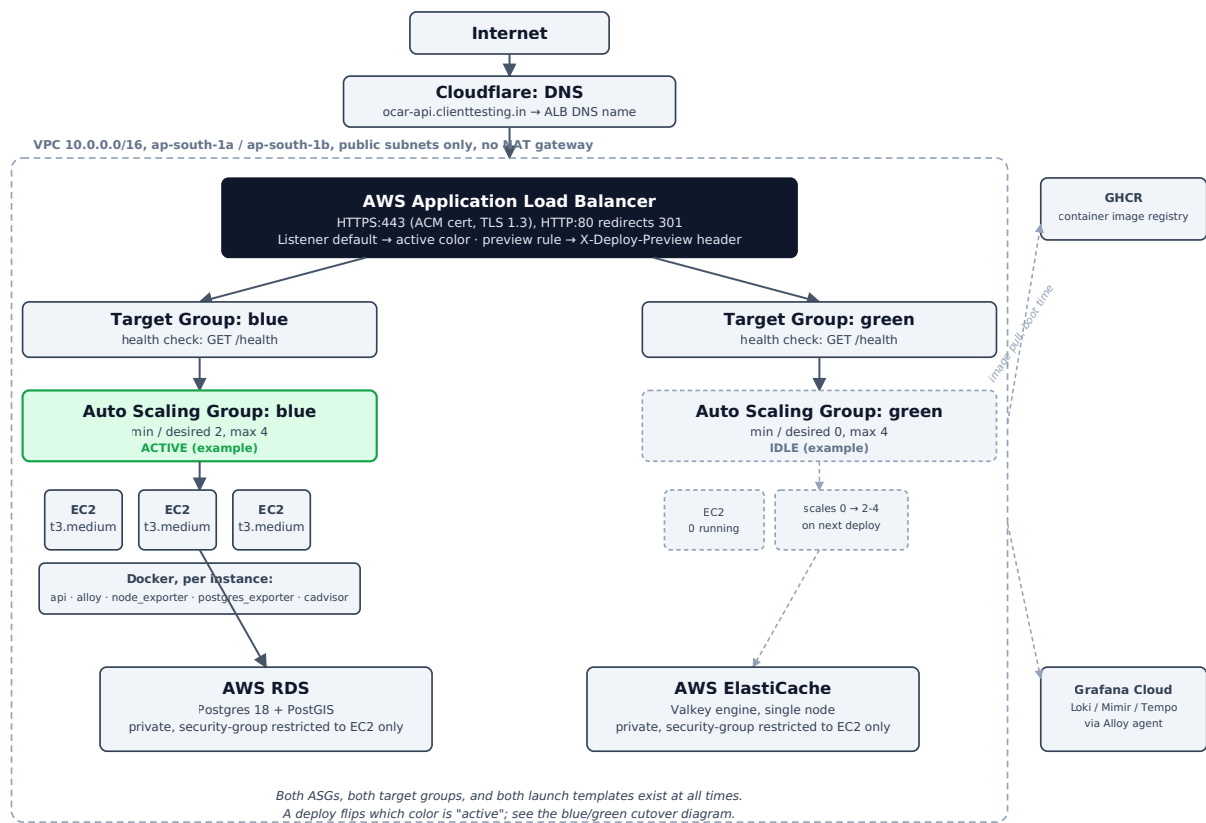
Version 1.0 | September 2026

This document describes the infrastructure and deployment pipeline as they exist today in the repository's Terraform configuration and GitHub Actions workflows, not as originally planned. Where an earlier plan changed during the actual build, this document reflects the change and says so directly. It replaces the older planning brief (`docs/TERRAFORM_INFRA_BRIEF.md`), which is now marked superseded and kept only for history.

1. Network and compute	3
2. Data layer	
3. Identity and access	
4. CI pipeline	
5. Deployment pipeline, overview	
6. Blue/green cutover, step by step	
7. Key architecture decisions	
8. Known gaps	

1. Network and compute

Everything runs in one AWS account, one region (`ap-south-1` , Mumbai), inside one VPC (`10.0.0.0/16`) with two public subnets, one per availability zone (`ap-south-1a` and `ap-south-1b`). There is no NAT gateway and no private subnet. EC2 instances sit in the public subnets but are not reachable from the internet directly: a security group only allows inbound traffic from the load balancer's own security group. Security is enforced at that layer, not by subnet placement. Section 7 explains why this was chosen over a NAT gateway and private subnets.



Network, compute, and data layer. Blue is shown active and green idle as an example; either can be the active color at any time.

1.1 Load balancer

One Application Load Balancer terminates TLS for the whole platform. The certificate comes from AWS Certificate Manager, DNS-validated (DNS itself lives on Cloudflare, not Route 53, so the validation record is added by hand once). The HTTPS listener uses the `ELBSecurityPolicy-TLS13-1-2-Res-PQ-2025-09` policy; the HTTP listener on port 80 only exists to issue a 301 redirect to HTTPS. `drop_invalid_header_fields` is enabled, which mitigates HTTP request-smuggling and desync attacks and should always be on for a public-facing ALB.

The listener's default action forwards to whichever color's target group is currently active. Two additional listener rules, one per color, match on a custom `X-Deploy-Preview` header and forward straight to that color's target group regardless of which one is live. This is how the deploy pipeline reaches the idle color directly through the real public ALB to smoke-test it,

before that color ever receives real traffic. A third rule, disabled by default and toggled by a Terraform variable, can serve a fixed 503 maintenance response for every request, for use if the API fleet itself is down and the application-level maintenance flag (which depends on Redis) cannot be reached.

1.2 Target groups

One target group per color. Health check: `GET /health`, healthy after 3 consecutive passes, unhealthy after 3 consecutive failures, 5-second timeout, checked every 15 seconds.

`deregistration_delay` is left at AWS's default of 300 seconds: this is how long the ALB keeps forwarding to a target that is being deregistered before force-closing its connections, which is what makes scaling the old color down after a deploy graceful rather than abrupt. A ride can run longer than any deregistration delay worth choosing, so this value bounds how much of the blast radius is covered, it does not need to cover every open connection. A WebSocket still open past that window gets closed, and the client's Socket.io library reconnects onto the new color automatically.

1.3 Auto Scaling Groups and instances

One Auto Scaling Group per color, both existing at all times so the idle one is instantly available for the next deploy. Whichever color is active runs at `min_size = desired_capacity = 2`, the idle color at 0. Both cap at `max_size = 4`. Scaling is on ALB request count per target (target value 1000), not CPU, since request volume reacts to load before CPU saturates on a request-heavy API. Health checks use the `ELB` type, meaning an instance is only considered healthy if it passes the target group's own health check, not merely if the instance is running. The health check grace period is 240 seconds, tuned for the observability sidecars' boot time.

Each instance is a `t3.medium` (burstable, 2 vCPU / 4 GB) running Ubuntu 24.04, launched from a Launch Template. IMDSv2 is required, with a metadata hop limit of 2 so a container on Docker's bridge network (one hop further than the host) can still reach instance metadata to inherit the instance's IAM role. The root volume is 20 GB, encrypted. Every instance runs identical containers via Docker Compose: the API, the Grafana Alloy agent, `node_exporter`, `postgres_exporter`, and `cAdvisor`. Nginx is not run on these instances; the ALB terminates TLS now, so it is no longer needed there.

Boot is fully automated and needs no SSH: the Launch Template's boot script installs Docker and the AWS CLI, reads the current image tag and the GHCR pull token from SSM Parameter Store, authenticates to GHCR, pulls every image, and starts the containers. The same script also keeps the Postgres exporter's database password in sync with RDS's auto-rotating Secrets Manager credential, on a 5-minute systemd timer, so a future password rotation cannot silently break metrics collection the way an earlier, hand-copied credential once did.

One naming detail worth knowing: the color that already existed before blue/green was introduced kept its original AWS resource names (no color suffix), because names are immutable on Auto Scaling Groups, target groups, and launch templates, and renaming would force a destroy-and-recreate of a live, traffic-serving resource. The newer color uses a clean `-green` suffix. This asymmetry is permanent and intentional, and it is mirrored exactly in the deploy workflow's own name computation.

2. Data layer

Postgres and the cache both moved onto AWS-managed services during this build. Production and staging previously used Neon (serverless Postgres) and an external Redis Cloud instance; both are now inside the same VPC as the application, reachable only from the EC2 security group.

2.1 Postgres

AWS RDS, Postgres 18.4, PostGIS enabled, instance class `db.t4g.small` (roughly 225 max connections, comfortable headroom over current peak usage). Storage is 20 GB gp3, encrypted, autoscaling up to 100 GB. Not Multi-AZ yet: a single instance, matching the general principle applied throughout this build of sizing honestly for current load rather than building in redundancy that is not yet needed. Turning on Multi-AZ later is a one-line Terraform change.

The master password is managed entirely by AWS: Secrets Manager creates and auto-rotates it, so the actual password value never sits in Terraform state or a `.tfvars` file. The application reads it directly from Secrets Manager via the instance's IAM role at connection time, not from a copied value. This replaced an earlier design where the password was hand-copied into an SSM parameter, which went stale after a rotation and caused a production outage on 2026-08-25 (see `docs/INCIDENT_2026-08-25_PROD_DB_AUTH_OUTAGE.md`). IAM database authentication is also enabled at the instance level for future use, but is deliberately not yet granted to the master role: that incident's post-mortem found that granting it to a role replaces password authentication for that role rather than adding to it, which is not a safe change to make to the master user.

The parameter group forces SSL on every connection (`rds.force_ssl`), preloads `pg_stat_statements`, enables logical replication (used for the Neon-to-RDS migration itself), and logs any query slower than 500ms to CloudWatch. Backups run daily with 7-day retention; deletion protection and a final snapshot on destroy are both on in production only.

2.2 Cache

AWS ElastiCache, Valkey engine 8.0, single node. This uses the replication-group resource type with one cluster and automatic failover turned off, rather than the simpler single-node cluster resource, because that simpler resource type does not yet support the Valkey engine in the current provider version. The practical outcome is the same either way: one primary, no replica. A custom parameter group forces `maxmemory-policy = noeviction`: BullMQ requires this, since job payloads must never be evicted under memory pressure, and the default policy on a fresh ElastiCache parameter group is `volatile-lru`, the exact misconfiguration that was already causing problems on the old external Redis Cloud instance. Both at-rest and in-transit encryption are enabled; the transit encryption mode is currently set to "preferred" rather than "required", which accepts both encrypted and unencrypted connections while every instance's configuration is confirmed to be using the encrypted form, avoiding a hard cutover that would break every connected client at once.

3. Identity and access

Nothing in this pipeline uses long-lived AWS access keys. GitHub Actions authenticates to AWS using OpenID Connect: GitHub issues a short-lived, signed token for each workflow run, and AWS trusts it directly through a federated identity provider. There is no static secret sitting in GitHub at all.

Three separate IAM roles exist for three separate purposes, each scoped to only what that purpose needs:

Role	Used by	Scope
EC2 instance role	Every running instance, at boot and continuously	Read exactly the SSM parameters it needs to boot (image tag, GHCR token, environment file, docker-compose and Alloy config), decrypt them, connect to RDS via IAM auth, and read the RDS master secret. Also carries AWS's managed SSM-agent policy, which is what lets the deploy pipeline route database migrations to a live instance over SSM instead of SSH.
Deploy role	The <code>deploy</code> job in <code>deploy.yml</code>	Read/write the per-color image-tag and active-color SSM parameters, scale and suspend/resume either color's Auto Scaling Group, describe target health and listeners, flip the one HTTPS listener (and only that listener, by ARN), read bake-window CloudWatch metrics, and route a migration command to a live instance. Nothing broader. The trust policy is scoped to this exact repository and keys off GitHub's environment-based claim format (<code>environment:production</code> or <code>environment:staging</code>), not a branch-ref claim, which is why the job declares a GitHub Environment at all.
Plan role	The <code>plan</code> job in <code>terraform-plan.yml</code> , PRs only	Broader read-only access across the services this configuration manages, plus read/write on the Terraform state object itself (scoped to the <code>prod/</code> key prefix). Never runs on a push to main, and never has write access to real infrastructure, only to the plan output.

4. CI pipeline

`ci.yml` runs on every push to any branch and on every pull request into `main` . Seven jobs run in parallel and must all pass:

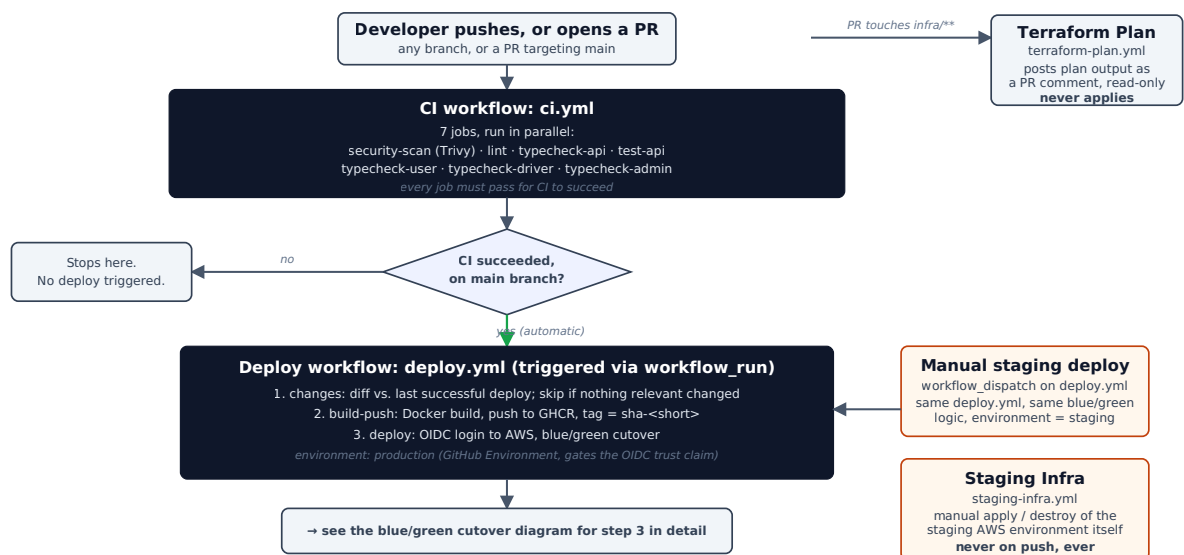
- **security-scan:** two Trivy passes. Dependency vulnerabilities are gated at CRITICAL only, since the monorepo currently carries a known set of pre-existing HIGH-severity transitive findings tracked separately rather than gated here. Misconfiguration and secret scanning is

gated at CRITICAL and HIGH; any accepted-risk finding is suppressed inline, at the resource, with a written reason.

- **lint**
- **typecheck-api, typecheck-user, typecheck-driver, typecheck-admin**: one per app in the monorepo
- **test-api**: spins up real Postgres (with PostGIS) and real Redis containers, runs migrations against them, then runs the full test suite against that real database and cache, not mocks

A separate workflow, `terraform-plan.yml`, runs only on pull requests into `main` that touch `infra/**`. It runs `terraform plan` and posts the output as a comment on the PR, updating the same comment on re-runs rather than posting a new one each time. It never applies anything.

5. Deployment pipeline, overview



Orange boxes: human-triggered only. Dark boxes: run automatically once their trigger condition is met.

How a push turns into a deploy, and where the two manual-only workflows sit outside that automatic path.

Four separate workflow files exist, each with a distinct trigger:

- `ci.yml`: automatic, on every push and PR, described above
- `terraform-plan.yml`: automatic, on PRs touching infrastructure code, described above
- `deploy.yml`: automatic for production, triggered whenever `ci.yml` completes successfully on `main`. Also supports a manual trigger, currently offered for staging only
- `staging-infra.yml`: manual only, never triggered by a push. Provisions or tears down the staging AWS environment itself, which is a separate action from deploying application code to it

Inside `deploy.yml`, three jobs run in sequence. The first, `changes`, decides whether a deploy is even necessary: it compares the commit CI just validated against the last commit this workflow successfully deployed, not simply the immediately preceding commit. That distinction matters

for a direct push of several commits to `main` at once: comparing only against the immediately preceding commit would miss everything except the very last one. If nothing under `api/`, the lockfile, or the deploy workflow itself changed since the last deploy, the pipeline stops here, so a documentation-only or infrastructure-only merge does not rebuild the image or restart the fleet.

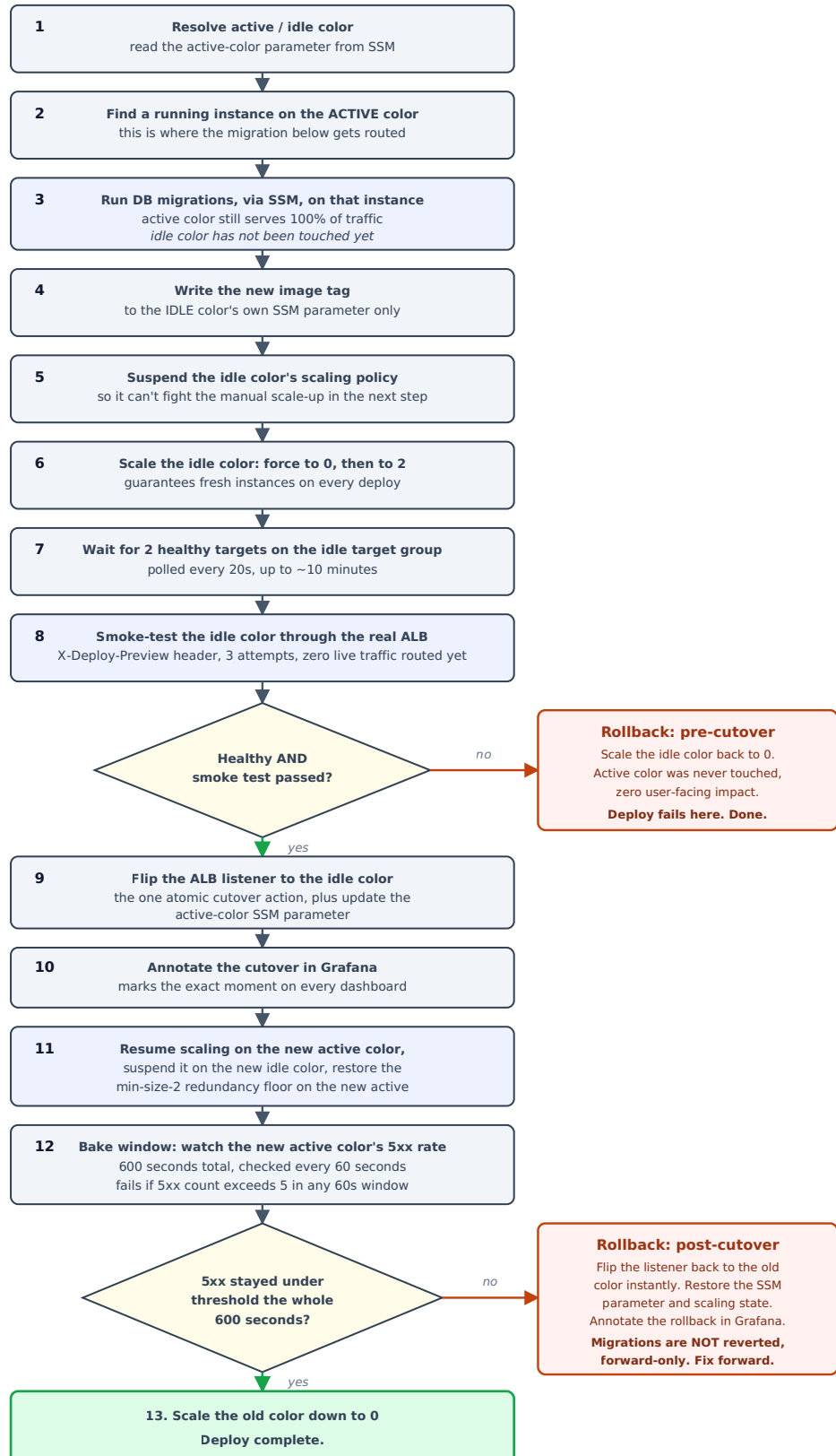
The second job, `build-push`, builds the Docker image from `api/Dockerfile` and pushes it to GitHub Container Registry, tagged with the short commit SHA. That SHA tag is what drives every downstream step; a `latest` tag is also pushed as a convenience for manual pulls, but nothing in the pipeline depends on it.

The third job, `deploy`, is the actual blue/green cutover, covered in full in the next section.

6. Blue/green cutover, step by step

This is what the `deploy` job actually does, in order. The two decision points are where a bad deploy gets caught, and each has its own rollback path.

Blue/green cutover: deploy.yml's "deploy" job, step by step



Total elapsed time, typical clean run: roughly 15-20 minutes from image push to old color scaled down, dominated by the 10-minute bake window plus the health-check poll, not by the cutover itself (a few seconds).

Two things are worth calling out explicitly. First, migrations run against the shared database before the idle color is touched at all, while the active color is still serving 100% of traffic, and they are never reverted on rollback: if a migration itself turns out to be the problem, the fix is a forward migration, not an automatic revert. Second, there are two genuinely different rollback moments, not one. A failure at step 8 costs nothing: the idle color never received real traffic, so rolling it back is invisible to users. A failure during the step 12 bake window is a real, brief production incident, since traffic was already flipped, but it is caught within a fixed 10-minute window and reversed by flipping the listener back, not by any slower mechanism.

7. Key architecture decisions

The choices below were made deliberately, with a reason, not by default. Each is listed with what was chosen, what the alternative was, and why.

Decision	Alternative considered	Why this was chosen
Blue/green cutover via an ALB listener flip	Rolling instance replacement (in-place)	Eliminates the window where old and new code run behind the same endpoint simultaneously. The cutover is a single, atomic API call against an endpoint that was already resolved once, not a gradual replacement.
ALB listener flip specifically, not a DNS-based switch	Route 53 weighted records or a CNAME swap between two load balancers	DNS resolves the domain to the ALB once, and that record never changes during a deploy. The actual routing decision lives one layer below DNS, inside the ALB's own listener rule, which is why the cutover is instant and immune to resolver caching or TTL propagation delay, a real weakness of DNS-based switching.
Terraform-native blue/green (two ASGs, two target groups, one listener flipped by the workflow)	AWS CodeDeploy	Every step is visible, version-controlled HCL and YAML, with no second deployment service's own state to keep in sync with Terraform's.
Public subnets, security enforced by security group, no NAT gateway	Private subnets behind a NAT gateway	A NAT gateway costs roughly \$32 to \$45 a month plus data transfer, for isolation that a correctly scoped security group already provides at this scale. Nothing on the internet can reach an instance directly regardless of subnet placement, since the EC2 security group only accepts traffic from the ALB's security group.
GitHub Actions OIDC federation, not static AWS access keys	Long-lived IAM user access keys stored as a repository secret	Credentials are short-lived and minted per workflow run. There is no static secret to leak, rotate, or accidentally commit.

Single-node RDS and single-node ElastiCache, no Multi-AZ or replica yet	Multi-AZ RDS, replicated ElastiCache, from day one	Sized for current load, not built ahead of demonstrated need. Both are a one-line Terraform change to add once real traffic justifies the added cost.
S3 backend with native locking for Terraform state	Local state, or S3 with a separate DynamoDB lock table	CI and a human operator's local machine need to read the same state. Native S3 locking is the current recommended approach and needs no separate DynamoDB table.
No Kubernetes	ECS or a Kubernetes control plane	ALB plus Auto Scaling Group plus Launch Template already provides automatic capacity and zero-SSH bootstrapping, without the operational overhead of running a control plane for a fleet that runs 2 to 4 nodes.
Account-wide singletons (the Terraform state bucket, the GitHub OIDC provider) split into a separate bootstrap module	Keep them in the same configuration as everything else	Both can only exist once per AWS account. Keeping them in the per-environment configuration meant every prod and staging apply tried, and failed, to recreate them.

8. Known gaps

These are open items, not hidden ones. Each has a specific, already-identified fix; none require new design work, only repo administrator access to execute.

No required-reviewer approval gate on production deploys. The `deploy` job already declares `environment: production`, which is what the OIDC trust policy relies on, but the "production" GitHub Environment itself has no required-reviewer rule configured. In practice, this means every push to `main` that passes CI deploys to production automatically today, with no human approval step in between. The fix is a repository-admin-only setting, not a code change.

Same gap on the staging Environment. `staging-infra.yml` references a "staging" GitHub Environment for the same gating purpose, but that Environment does not exist yet either. A `destroy` dispatch against staging is currently unguarded by any approval step.

No branch protection on `main`. Nothing currently stops a direct push to `main`, bypassing a pull request and its required CI checks, from landing code that then deploys automatically per the gap above.

All three are blocked purely on repository administrator access, which the engineering team does not currently hold on this repository. Exact commands to close each gap are already documented and ready to run.